singleton, while leaving the `LifetimeDemoService` registered with a transient lifetime:

```
builder.Services.AddTransient<LifetimeDemoService>();
builder.Services.AddSingleton<SingletonService>();
```

Change the markup in the Razor page to output the following:

```
<p>The singleton service's transient dependency returned @Model.Value</p>
```

Run the page, and refresh it. Notice that the value never changes. You don't get a new instance of the `LifetimeDemoService` on each request because the consumer's constructor is not being called, since it is a singleton.

7.2.4 Other service registration options

The aforementioned examples use one of the `Add[LIFETIME]` methods that takes two generic arguments—the first representing the service type and the second representing the implementation. This is the pattern you are likely to use most often. We have also looked at the version of the `Add[LIFETIME]` method that takes an implementation. Here we'll review some other registration options that provide additional capabilities.

Imagine your `SimpleCityService` needs some constructor arguments passed to it. You can do this by passing in a factory that defines the arguments to be passed:

```
builder.Services.AddTransient<ICityService>(provider => new
➥ SimpleCityService(args));
```